

# LLMProvider

## LLMProvider

一个接口，43家供应商——内置熔断、重试与健康监测机制。

### 概要

LLMProvider 是一个通用的、可复用的 Go 模块，定义了统一的 LLMProvider 接口及其生产级弹性模式——熔断器、健康监测、带退避的重试、延迟加载——并在此统一契约下提供 43 个具体供应商实现，配备兼容 OpenAI 的通用适配器，且采用真实、无硬编码回退的模型发现机制。

### 简要描述

一个可复用的 Go 模块，提供单一 LLMProvider 接口（Complete、CompleteStream、HealthCheck、GetCapabilities、ValidateConfig），并集成容错基础设施——熔断器、健康监测、带抖动退避的重试、延迟初始化——覆盖 43 个供应商适配器及兼容 OpenAI 的通用适配器。线程安全。

### 详细描述

LLMProvider 是每个消费 LLM 的服务都需要却鲜有人构建完善的抽象层——那种将演示代码与能承受真实流量的系统区分开来的基础设施。它定义了一个统一的、能力感知的接口——Complete、CompleteStream、HealthCheck、GetCapabilities、ValidateConfig——使应用代码无论调用哪个后端（共 43 个），都只需面向同一个契约。同时，它还提供了将脆弱的供应商调用转化为可安全上线的生产级组件所需的运维强化机制。

一个三态熔断器（闭合→开启→半开启）透明地封装任意供应商——包括其流式通道，其中空流会被正确识别为失败——确保单个行为异常的后端触发熔断后，不会拖垮整个服务；中央 CircuitBreakerManager 实时跟踪所有熔断器状态。可配置的健康监测器持续评估供应商的健康状态（健康/降级/不健康/未知），基于阈值和间隔检查，使性能下降能被实时观测，而非等到故障发生才发现。重试逻辑在指数退避基础上叠加抖动机制，智能判断重试时机——仅重试值得重试的错误（429、5xx、瞬时网络故障），绝不浪费资源在

4xx 错误或已取消的上下文上，且延迟时间受限，避免退避风暴失控。延迟初始化模式则将每个供应商的构建推迟至首次实际使用时——这一设计刻意降低了注册全部 43 个供应商的开销。

该模块提供 43 个具体供应商包，以及一个兼容 OpenAI 的 generic 通用适配器，后者可针对任意 `/v1/chat/completions` 端点实现完整接口——支持 Bearer 认证、SSE 流式传输及正确的 [DONE] 处理——因此，即使某供应商未提供专用包，只要将适配器指向其 URL 端点，即可立即成为一等公民。凭证统一通过 `apikeys` 解析（严格遵循 `ApiKey_<Provider>` 命名规范），从根源上杜绝了“测试套件通过硬编码密钥，实际密钥未接入，生产环境崩溃”这一类问题。模型发现机制极为严谨：通过 TTL 缓存查询实时供应商 API，且——根据治理要求——旧的硬编码回退方案已被彻底移除。当实时发现失败时，LLMProvider 宁可返回空结果，也不提供过时的目录，确保调用方绝不会收到看似有效却无法调用的模型 ID。所有组件均为线程安全设计，支持并发使用。

## 我们为何构建它

生产环境中的 LLM 原始调用屡屡失效——供应商限流、服务降级或宕机，一个糟糕的后端便能拖垮整个服务。模型目录漂移不定，硬编码的列表却仍在分发早已失效的调用 ID。LLMProvider 将接口、弹性模式与真实发现机制集中化，让每个消费者免费获得容错能力与可靠性。

## 为何它是变革者

它将“集成 LLM 供应商”简化为一步到位——实现一个接口，或直接将通用适配器指向某个端点——随后自动且透明地为该供应商包裹断路器、健康监测与抖动退避重试机制。弹性不再是每个团队在首次故障后（在截止日期压力下拙劣地）重复发明的轮子，而是成为库中默认行为，覆盖全部 43 个后端。可靠性工程只需编写一次，经过严格测试，便能被所有引入者免费继承。

## 创新之处

- 统一能力感知接口——补全、流式传输、健康检查、能力查询与配置验证，全部整合为每个后端一致遵循的单一契约。
- 透明断路器包装——包括流式传输。断路器不仅保护请求/响应，还保护 `CompleteStream` 的通道，并将空流视为真正的失败——通过无死锁、解锁状态下的监听器通知机制实现。
- **43 个供应商包 + 通用 OpenAI 兼容适配器**——专用包保持轻量，任何未列入清单但支持 `/v1/chat/completions` 的供应商，只需将适配器指向其端点即可立即生效。
- 单一凭证管理（`apikeys`）——仅有一处读取 `ApiKey_<Provider>` 环境变量，从结构上消除“测试通过、产品崩溃”的不一致问题，而非仅仅发出警告。

- **真实模型发现（无硬编码回退）**——供应商 API 实时更新，缓存基于 TTL；失败时返回 nil，绝不提供过期或虚构的目录，避免分发无法调用的 ID。
- **惰性初始化（`sync.Once`）**——构造延迟至首次使用，注册全部 43 个供应商的成本几乎为零，直到实际调用时才产生开销。
- **反虚假、多地域挑战栈**——真实运行器在五个地域测试断路、健康检查与重试行为，并通过配对变异测试（未变异代码必须正常退出；注入变异必须强制退出 99）进行验证，确保测试通过即代表行为可靠。

## 最大技术挑战及解决方案

- **级联供应商故障。** 一个不稳定的后端绝不能拖垮整个服务。解决方案：采用三态断路器（闭合→开启→半开启），透明包装任何供应商及其流式传输，持续故障时触发开启，半开启状态下探测恢复，并由 `CircuitBreakerManager` 集中协调。
- **瞬时错误与限流。** 解决方案：基于状态感知的指数退避加抖动—— $\min(\text{初始延迟} \times \text{倍率}^{(n-1)}, \text{最大延迟}) \pm \text{抖动}$ ，避免重试同步化形成“惊群效应”。仅重试应重试的错误（429、500、502、503、504 及网络错误），拒绝在上下文取消或其他 4xx 错误上浪费尝试。
- **大量注册但未使用的供应商。** 43 个供应商注册后，实际服务中仅少数被调用，提前构造纯属浪费。解决方案：通过 `sync.Once` 实现惰性初始化，仅实际调用的供应商才承担初始化成本。
- **分发无效模型 ID。** 解决方案：彻底移除硬编码的发现回退层（依据 `CONST-036`），实时发现失败时直接返回空值——同时采用防御性复制返回，确保调用者无法修改缓存或与其他读取者竞争。真实性通过结构而非约定强制保障。
- **流式传输与并发正确性。** 隐蔽的故障模式是断路器锁与监听器回调之间的死锁。解决方案：快照监听器，在解锁状态下以 5 秒超时通知，并在重置时先解锁再通知——所有组件均为并发设计，并通过 `-race` 测试套件验证。

内容

## 技术栈

- **Go (1.25.3)**——选用原因：一流的并发支持、静态二进制构建及强大的标准库；包含模块、接口、所有弹性原语及全部 43 个适配器。
- **net/http**（标准库）——纯粹无依赖的 HTTP 实现：为各供应商客户端、通用 OpenAI 兼容适配器及实时发现调用提供动力，无需审计或修补第三方传输层。
- **logrus**——结构化、分级日志记录，精准覆盖运维人员关注的核心区域：熔断器状态切换及发现路径。
- **testify**——驱动测试套件，尤为关键的是，通过变异分支固定确保测试通过的真实性。
- **yaml.v3**——解析 i18n 语言包及配置文件，格式保持人工可编辑性。
- **digital.vasic.models**——共享的 `LLMRequest/LLMResponse/ProviderCapabilities` 类型，统一存放以确保所有适配器使用相同的术语体系（为文档化的运行时依赖）。

- 第一方包——circuit（熔断）、health（健康检查）、retry（重试）、apikey（API密钥）、discovery（发现）、providers/（43家供应商 + generic通用适配器）及i18n（国际化）：将弹性与集成层拆分为独立可测的小单元，避免单体架构。
- .env + ~/api\_keys.sh (ApiKey\_<Provider>命名规范) ——单一明确的凭证来源，确保测试与生产环境的密钥配置方式一致。
- Makefile竞态测试套件（-race -p 1） + 挑战测试运行器——反虚假验证的核心支柱：竞态检测器验证并发正确性，挑战测试运行器通过混沌、DDoS、扩缩、压力、实时发现及无中断场景，全面验证真实行为。

## 状态与诚信说明

- 状态：**Beta**。解耦可复用模块；GitHub代码库已公开。
- 许可证：待定。 存在不一致——doc.go声明为MIT，但LICENSE文件采用Apache-2.0风格——发布前需确认。
- LLMsVerifier为上游模型目录的唯一权威来源。helix-deps.yaml清单已过时（声明deps: []，但文档明确依赖digital.vasic.models）；发现模块的“Tier 2 (models.dev)”为计划中的占位符，尚未启用。

优先级：Helix-主（LLM-基础设施集群——解耦可复用模块）。优先级低于HelixTrack。